



Experiment – 2

Name: Simranjeet Kaur

UID: 24BAI70048

Branch: CSE – AIML (CS221)

Section/Group: 24AIT_NTPP-2A

Semester: 5th

Date of Performance: 17/07/2026

Subject: Full Stack-II

Subject Code: 24CSP-337

EXPERIMENT – 2.1

1. AIM: To design and implement a centralized state management system using Redux Toolkit for managing posts and platform-related data.

2. OBJECTIVE:

1. To understand concept of global state management.
2. To implement Redux Toolkit for scalable state handling.
3. To manage asynchronous data flows.

3. SOFTWARE REQUIREMENTS:

1. Windows/Linux/macOS
2. Node.js (LTS Version)
3. NPM (Node Package Manager)
4. Visual Studio Code
5. Google Chrome / Microsoft Edge
6. React.js
7. Command Prompt / Terminal
8. Redux Toolkit

4. THEORY

React is an open-source JavaScript library developed by Meta (Facebook) for building fast, dynamic, and interactive user interfaces. It follows a component-based architecture, where the user interface is divided into reusable components. This approach improves code reusability, readability, and maintainability.

As frontend applications grow in complexity, managing state across multiple components becomes increasingly challenging. Traditional approaches such as prop drilling lead to tightly coupled components and reduced maintainability. Redux provides a centralized store that acts as a single source of truth, enabling predictable state transitions through actions and reducers. Redux Toolkit simplifies Redux implementation by reducing boilerplate and providing built-in utilities for common tasks. State normalization is an important concept where data is stored in a structured format, reducing redundancy and improving performance. This is especially useful when managing collections such as posts or users. Additionally, Redux supports handling asynchronous operations, enabling integration with APIs or simulated mock data sources.

5. IMPLEMENTATION/PROCEDURE

1. Install Redux Toolkit and React-Redux using npm.
2. Configure the Redux store using `configureStore()`.
3. Create slices for posts and platforms using `createSlice()`.
4. Define the initial state structure for the application.
5. Implement reducers for CRUD operations on posts.
6. Connect React components to the Redux store using `useSelector()` and `useDispatch()`.
7. Manage asynchronous data flows using `createAsyncThunk()` and mock API calls.

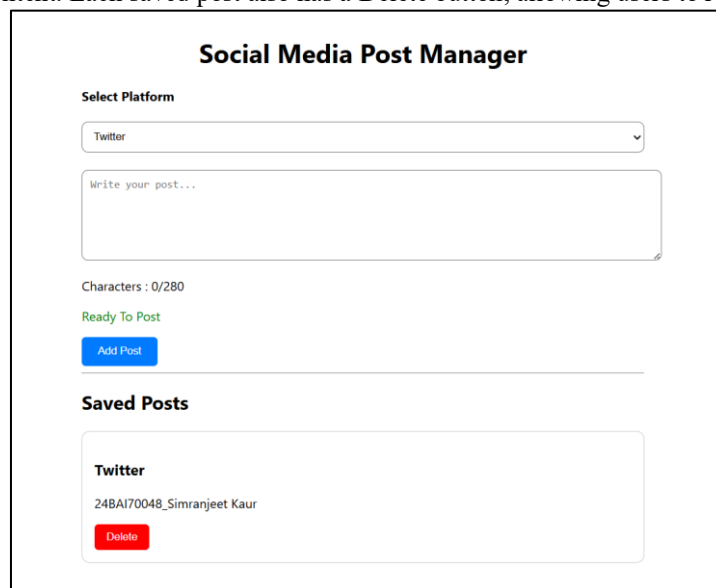
6. CODE

```
import React from "react";
import PostComposer from "../components/PostComposer";
import DraftList from "../components/DraftList";
function App() {
  return (
    <div
      style={{
        maxWidth: "700px",
        margin: "40px auto",
        padding: "20px",
      }}
    >
      <PostComposer />
      <DraftList />
    </div>
  );
}

export default App;
```

7. OUTPUT

The output presents a Social Media Post Manager interface developed to manage posts across different social media platforms. The user can select a platform, enter post content in the text area, and view the character count, which is currently limited to 280 characters for Twitter. The application displays a “Ready To Post” status when the content is valid and provides an “Add Post” button to save the post. Added posts are shown in the Saved Posts section along with the selected platform name and post content. Each saved post also has a Delete button, allowing users to remove posts from the list.



The screenshot shows a web application titled "Social Media Post Manager". It features a "Select Platform" dropdown menu with "Twitter" selected. Below this is a text input field labeled "Write your post...". Under the input field, it shows "Characters : 0/280" and a green status "Ready To Post". A blue "Add Post" button is located below the status. At the bottom, there is a "Saved Posts" section. It contains a card for "Twitter" with the text "24BAI70048_Simranjeet Kaur" and a red "Delete" button.

Fig 2.1: Redux Toolkit-Based Centralized State Management for Social Media Posts.

8. LEARNING OUTCOME

- Understand centralized state management using Redux Toolkit.
- Configure and manage Redux store.
- Create slices for managing posts and platform data.
- Implement CRUD operations using reducers.
- Use React state management to handle user input and dynamic UI updates.
- Connect React components with Redux store using hooks.

EXPERIMENT –2.2

1. **AIM:** To optimize state access and improve application performance using memoized selectors and efficient rendering strategies.

2. **OBJECTIVE:**

1. To understand the concept of derived state
2. To implement memoized selectors using Reselect
3. To reduce unnecessary component re-renders

3. **SOFTWARE REQUIREMENTS:**

1. Windows/Linux/macOS
2. Node.js (LTS Version)
3. npm (Node Package Manager)
4. Visual Studio Code
5. Google Chrome / Microsoft Edge
6. React.js
7. Command Prompt / Terminal
8. Redux Toolkit

4. **THEORY**

In large-scale applications, inefficient state access and frequent re-renders can significantly degrade performance. Redux selectors are used to extract specific pieces of state; however, without optimization, they may recompute values unnecessarily. Memoization is a technique used to cache computed results and return them only when input data changes. Libraries like Reselect provide memoized selectors that prevent redundant computations and reduce rendering overhead. Derived state refers to data computed from the existing state rather than stored directly. By using selectors, applications can compute complex data efficiently without duplicating state. Optimizing selectors and rendering behavior ensures that components update only when necessary, improving responsiveness and scalability.

5. **IMPLEMENTATION/PROCEDURE**

1. Define basic selectors to access posts and platform data from the Redux store. Implement the Save Draft functionality using `useState()`.
2. Implement memoized selectors using `createSelector()` from Reselect. Implement Edit Draft functionality.
3. Compute derived data such as filtered and grouped posts using selectors.
4. Integrate memoized selectors into React components using `useSelector()`.
5. Use `React.memo` and `useMemo` to reduce unnecessary component re-renders.
6. Analyze component rendering behavior and identify unnecessary re-renders.
7. Optimize state access and rendering to improve application performance.

6. **CODE**

```
import React from "react";
import PostComposer from "./components/PostComposer";
import DraftList from "./components/DraftList";
import Stats from "./components/Stats";
function App() {
  return (
    <div
      style={{
        maxWidth: "700px",
        margin: "40px auto",
        padding: "20px",
      }}
    >
      <PostComposer />
```

```

    <hr />
    <Stats />
    <DraftList />
  </div>
);
}

```

```
export default App;
```

7. OUTPUT

The output shows a Social Media Post Manager developed using React and Redux Toolkit. Users can select a social media platform, write a post, and view the character limit according to the selected platform. The application also displays statistics using memoized selectors, including total posts and platform-wise post counts. Saved posts are displayed in the Saved Posts section, where users can view the platform and content and delete posts when required. This demonstrates centralized state management, derived state, memoized selectors, and efficient rendering using Redux Toolkit and React.

Social Media Post Manager

Select Platform

Instagram

Write your post...

Characters : 0/2200

Ready To Post

Add Post

Statistics

Total Posts: 1

Twitter: 1

LinkedIn: 0

Instagram: 0

Saved Posts

Twitter

Simranjeet Kaur_24BAI70048

Delete

Instagram

Simran_24BAI70048

Delete

Fig 2.2: Memoized selectors for optimized state access.

8. LEARNING OUTCOME

- Understand derived state in Redux.
- Optimize state access and data computation.
- Reduce unnecessary component re-renders.
- Design scalable and maintainable Redux application.
- Analyze rendering efficiency in React application.
- Improve application performance.